



LxCameraSDK (Linux)

示例程序使用说明

版本号: V2.0

发布日期: 2026.07.27

GitHub: <https://github.com/Lanxin-MRDVS/CameraSDK>

知识库: <https://hub.mrdvs.cn/>

官网: <https://www.mrdvs.cn/> & <https://mrdvs.com/>

浙江迈尔微视科技有限公司 版权所有



目录

1 文档概述	1
1.1 文档说明	1
1.2 SDK 内容	1
1.3 系统适用范围	1
2 首次运行准备	2
2.1 安装基础依赖	2
2.2 安装 SDK	2
2.3 配置相机网络	3
3 C/C++ 示例	4
3.1 批量编译	4
3.2 示例功能使用	5
3.3 单独编译	6
3.4 内置算法模式	6
4 Python 示例	7
4.1 安装 wheel	7
4.2 运行基础示例	7
5 ROS1 示例	8
5.1 安装 ROS Noetic 及依赖	8
5.2 编译并启动普通相机节点	9
5.3 launch 文件	10
5.4 普通节点参数	11
5.5 话题与服务	12
6 ROS2 示例	12
6.1 安装 ROS2 Humble 及依赖	12
6.2 编译、启动与验证	13
6.3 CycloneDDS	14
7 视觉定位、建图与重定位	15
7.1 输入、输出与关键配置	15
7.2 联调顺序	15
8 常见问题	16

1 文档概述

1.1 文档说明

本文用于指导开发人员在 Linux 系统中完成 LxCameraSDK 安装、相机网络检查以及 C/C++、Python、ROS1 和 ROS2 示例的编译与运行。首次使用建议先完成第 2 章和第 3 章，再按实际开发环境选择后续章节。

***功能支持：**数据流、IMU、雷达点云和内置算法由相机型号、固件及算法授权共同决定。程序返回 LX_W_NOT_SUPPORT 时，表示当前设备不支持该功能，不代表 SDK 安装失败。

1.2 SDK 内容

目录或文件	内容	用途
install.sh	SDK 安装脚本	将头文件和对应架构动态库安装到/opt/MRDVS。
set_socket_buffer_size.sh	网络缓存脚本	设置 Linux Socket 收发缓存上限，install.sh 会自动调用。
set_firewall.sh	防火墙脚本	依赖广播搜索工具；未提供对应 Tool 时改用手动规则。
SDK/include	C/C++头文件	接口、特征枚举、数据结构和应用算法结构体。
SDK/lib	Linux 动态库	分别提供 x86_64、32 位 ARM 和 64 位 ARM 版本。
Sample/C	C/C++示例	基础取流、回调、多相机、IMU、雷达点云和内置算法。
Sample/python	Python wheel	Python 封装及 wheel 内部示例。
Sample/ROS	ROS1 示例	普通相机、雷达点云、视觉定位和传感器模拟节点。
Sample/ROS2	ROS2 示例	ROS2 版本的相机、定位和传感器模拟节点。

1.3 系统适用范围

示例	推荐环境	说明
C/C++	Linux x86_64、arm、aarch64	依赖 gcc/g++、CMake 和 OpenCV；OpenCV 用于图像显示与保存，动态库必须与 CPU 架构一致。
Python	Python3	wheel 包含 numpy、OpenCV 依赖；示例使用图形

示例	推荐环境	说明
		界面时需要桌面环境。
ROS1	Ubuntu 20.04 + ROS Noetic	软件包同时保留 Kinetic (Ubuntu16.04)、Melodic (Ubuntu18.04) 兼容说明；Noetic 已结束官方维护，仅用于存量系统。
ROS2	Ubuntu 22.04 + ROS2 Humble	不建议在 Ubuntu 20.04 上使用该 ROS2 示例。

2 首次运行准备

2.1 安装基础依赖

在 Ubuntu 20.04 或 22.04 的全新系统中，先安装 C/C++、图像处理和 Python 示例所需组件。OpenCV 不是 C/C++ 示例编译的强制依赖，但建议安装，以启用图像显示和保存功能。

```
# 更新软件源并安装 C/C++ 编译及图像处理依赖
```

```
sudo apt update
```

```
sudo apt install -y build-essential cmake pkg-config libopencv-dev
```

```
# 安装 Python 运行及图像处理依赖
```

```
sudo apt install -y python3 python3-pip python3-numpy python3-opencv
```

安装完成后检查系统和工具版本：

```
# 检查系统架构和 Ubuntu 版本
```

```
uname -m
```

```
lsb_release -ds
```

```
# 检查主要编译及运行工具
```

```
gcc --version
```

```
g++ --version
```

```
cmake --version
```

```
pkg-config --modversion opencv4
```

```
python3 --version
```

2.2 安装 SDK

进入包含 install.sh 的 MRDVS 目录后执行安装。脚本会删除旧的 /opt/MRDVS 并重新安装，因此该目录内如有自定义文件，应先备份。

```
# 安装 SDK 文件及系统参数
```

```
sudo bash install.sh
```

```
# 注册 SDK 动态库路径并刷新系统缓存
```

```
echo /opt/MRDVS/lib | sudo tee /etc/ld.so.conf.d/mrdvs.conf
```

```
sudo ldconfig
```

使用系统动态库缓存可避免仅在当前终端设置 LD_LIBRARY_PATH。安装完成后执行以下检查：

```
# 检查头文件和主要动态库是否已安装
test -f /opt/MRDVS/include/lx_camera_api.h && echo "header: OK"
test -f /opt/MRDVS/lib/libLxCameraApi.so && echo "camera library: OK"
test -f /opt/MRDVS/lib/libLxDataProcess.so && echo "data library: OK"
# 检查动态库注册结果及依赖项
ldconfig -p | grep LxCameraApi
ldd /opt/MRDVS/lib/libLxCameraApi.so
```

*架构：uname -m 为 x86_64 时使用 linux_x64 库；包含 arm 时使用 linux_arm32 库；包含 aarch64 时使用 linux_aarch64 库。若脚本提示 not found matched platform，应停止运行并确认软件包是否提供目标架构动态库。

2.3 配置相机网络

1. 将相机与主机有线网卡直连，或接入同一交换机。主机网卡与相机必须处于同一 IPv4 网段，且不能使用相同的 IP 地址。
2. 需要同时访问互联网时，应使用 Wi-Fi 或另一块网卡连接外部网络。只有一块网卡且与相机直连时，无法同时访问互联网属于正常现象，应在连接相机前完成在线依赖安装。
3. 首次验证建议使用独立有线网卡，并按以下示例配置：相机 IP 地址为 192.168.100.82，主机 IP 地址为 192.168.100.10，子网掩码为 255.255.255.0，网关留空。若相机已配置为其他 IP 地址，无需强制修改为上述地址，只需将主机网卡设置到相机所在网段。例如，相机 IP 地址为 192.168.1.20 时，主机可设置为 192.168.1.10。
4. 软件包中的 Python 基础示例默认连接 192.168.100.82。采用上述推荐配置可减少首次运行时的参数调整；若相机使用其他 IP 地址，应按第 4.2 节修改连接方式。C/C++ 基础示例通过设备搜索结果连接，不要求相机使用固定 IP 地址。
5. 确认网卡已连接且路由正确后，再运行示例程序。同一台相机同一时间只允许一个客户端取得控制权限，运行新示例前应关闭其他示例程序、ROS 节点和上位机。

```
# 检查网卡地址和路由
ip address
ip route
# 检查系统防火墙状态
sudo ufw status
```

UFW 处于 active 状态且设备搜索或取流受阻时，按网络安全策略开放 SDK 使用的入站端口：

```
# 开放设备搜索、控制和数据传输所需端口
sudo ufw allow 9700/udp
sudo ufw allow 9800/udp
sudo ufw allow 3956/udp
sudo ufw allow 3959/udp
sudo ufw allow 31900/udp
sudo ufw allow 32000/udp
sudo ufw allow 39560/udp
sudo ufw allow 9900/tcp
# 重新加载防火墙规则
sudo ufw reload
```

***防火墙脚本：**set_firewall.sh 需要 Tool 目录中的 lx_boardcast_tool。软件包未提供该工具时不要依赖脚本自动获取相机 IP，使用上面的固定端口规则或由网络管理员按相机 IP 限制来源。

3 C/C++示例

3.1 批量编译

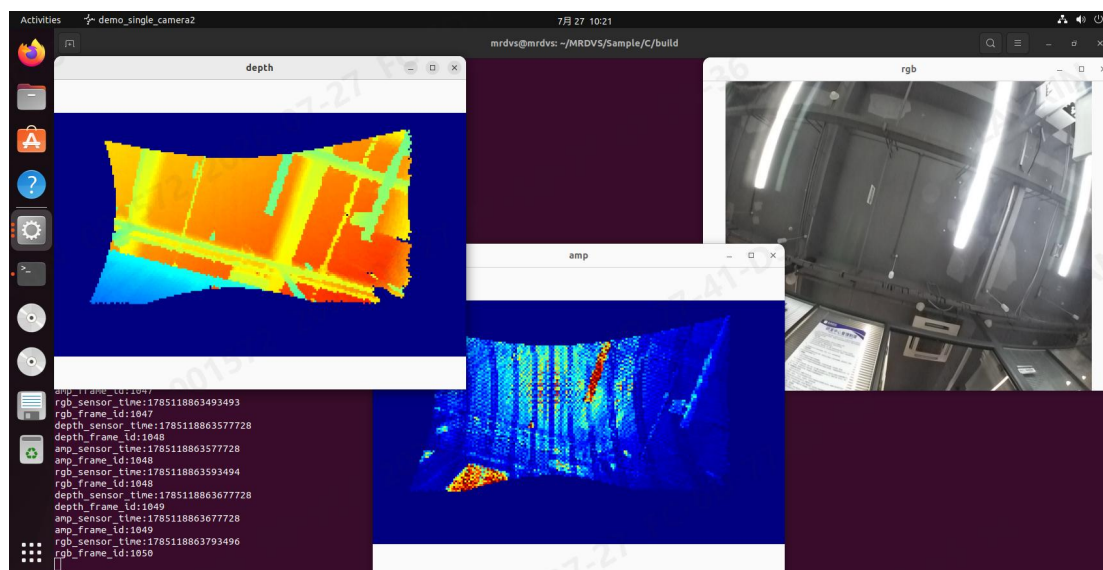
在包含 install.sh 的 MRDVS 目录中执行以下命令，可一次编译除 arm_local_camera 外的 9 个示例：

```
# 进入 C/C++示例目录并创建独立编译目录
cd Sample/C
mkdir -p build
cd build
# 配置并编译全部示例
cmake ..
make -j
# 查看生成的可执行程序
ls bin
```

首次验证优先运行单相机同步取流示例。程序搜索设备、按索引 0 打开第一台相机并输出 API 版本、设备信息、帧时间戳和图像信息；输入 q 后回车退出。

```
# 运行单相机基础取流示例
./bin/demo_single_camera2
```

***成功判断：**日志中出现 DcGetDeviceList success、设备 IP/序列号以及连续的 depth、amp 或 rgb 帧信息，即表示 SDK 安装、网络和基础取流链路正常。是否出现某一类数据取决于相机当前的数据流配置。



3.2 示例功能使用

示例	功能	使用与退出
demo_single_camera2	同步读取 FrameInfo、深度、强度、RGB 和 XYZ 点云；可保存 depth.pgm、amp.pgm、rgb.png 及 xyz.pcd。	基础验证首选；启用 OpenCV 编译时，输入 s 保存或覆盖当前的 depth.pgm、amp.pgm、rgb.png 和 xyz.pcd，输入 q 后回车退出。
demo_frame_callback	注册帧回调，在回调中处理 FrameInfo 及算法结果。	回调内不宜执行耗时操作；输入 q 后回车退出。
demo_multi_cameras	枚举全部设备，每台相机独立线程取流并输出状态和帧率。	需评估网卡和交换机带宽；输入 q 后回车退出。
demo_imu_callback	注册 IMU 回调，输出时间戳、加速度、角速度和计算频率。	仅支持带 IMU 型号；输入 q 后回车退出。
demo_lidarpoint_callback	获取 XYZIRT 雷达点云结构体，并可保存 PCD 文件。	仅支持雷达点云型号；输入 s 保存、输入 q 退出。
demo_application_obstacle_v1	启用避障 V1，周期读取避障 IO 和区域索引。	仅支持对应算法型号；程序持续运行，使用 Ctrl+C 结束。

示例	功能	使用与退出
demo_application_obstacle_v2	启用避障 V2，输出地面、点云和障碍框信息。	仅支持对应算法型号；程序持续运行，使用 Ctrl+C 结束。
demo_application_pallet	启用托盘对接算法，输出 x、y 和 yaw。	仅支持托盘算法型号；程序持续运行，使用 Ctrl+C 结束。
demo_application_location	演示视觉定位、地图导入导出和外部传感器数据传入。	二次开发样例，运行前必须配置地图、里程计、外参和算法参数。
demo_arm_local_camera	在相机内部 ARM 系统通过 127.0.0.1 调用 SDK。	仅用于相机内部 ARM 环境；不在批量编译目标中。

3.3 单独编译

single_camera2、multi_cameras、frame_callback、application_pallet 和 arm_local_camera 目录提供独立 Makefile。以 single_camera2 为例：

```
# 从批量编译目录进入单相机示例目录
cd ../single_camera2
# 编译并运行当前示例
make
./demo_single_camera2
```

***application_location:** 独立 Makefile 可以完成基础编译，但不会启用 OpenCV 显示功能；需要图像显示时，应使用顶层 CMake 编译。运行前还需把地图导入文件、里程计输入和定位参数改为项目实际配置，不能直接使用示例数据。

3.4 内置算法模式

值	枚举	用途
0	MODE_ALL_OFF	关闭内置应用算法。
1	MODE_AVOID_OBSTACLE	避障 V1。
2	MODE_PALLET_LOCATE	托盘对接。
3	MODE_VISION_LOCATION	视觉定位。
4	MODE_AVOID_OBSTACLE2	避障 V2。
5	MODE_GENERIC_DATA	应用算法平台自定义数据。

***设备占用：**示例正常退出时会调用 DcStopStream 和 DcCloseDevice。若使用 Ctrl+C 或进程异常结束后立即出现 LX_E_CTRL_PERMISS_ERROR，应确认没有残留进程，并等待相机控制心跳释放后再启动。

4 Python 示例

4.1 安装 wheel

进入 MRDVS/Sample/python 目录安装 Python 封装。系统已通过 apt 安装 numpy 和 OpenCV 时，使用 --no-deps 可避免重复下载大型软件包。

```
# 进入 Python 示例目录并安装 SDK 封装
cd Sample/python
python3 -m pip install --user --no-deps ./lx_camera_py-1.3.3-py3-none-any.whl
# 检查 Python 模块是否可以正常导入
python3 -c "import LxCameraSDK; print('LxCameraSDK import: OK')"
```

4.2 运行基础示例

wheel 内置 single_camera、application_obstacle、application_pallet、application_location 示例。安装包中的 single_camera 默认使用指定的动态库名称和相机 IP 地址。为适配不同的安装位置和相机网络，首次运行前应将示例复制到当前目录，并改为使用 SDK 的实际安装路径及设备索引。

```
# 将已安装的基础示例复制到当前目录
python3 -c "import inspect, shutil, LxCameraSDK.Sample.single_camera as d;
shutil.copy(inspect.getsourcefile(d), 'single_camera.py')"
# 改为按搜索列表索引打开第一台相机
sed -i 's@open_mode = LX_OPEN_MODE.OPEN_BY_IP@open_mode =
LX_OPEN_MODE.OPEN_BY_INDEX@' single_camera.py
# 使用 SDK 的实际安装路径加载动态库
sed -i
's@main("libLxCameraApi.so")@main("/opt/MRDVS/lib/libLxCameraApi.so")@'
single_camera.py
# 确认连接方式和动态库路径已正确修改
grep -nE 'open_mode =|open_param = "0"|main\(' single_camera.py
```

确 认 输 出 中 包 含 OPEN_BY_INDEX 、 open_param="0" 和 /opt/MRDVS/lib/libLxCameraApi.so 后，运行示例：

```
python3 single_camera.py
```

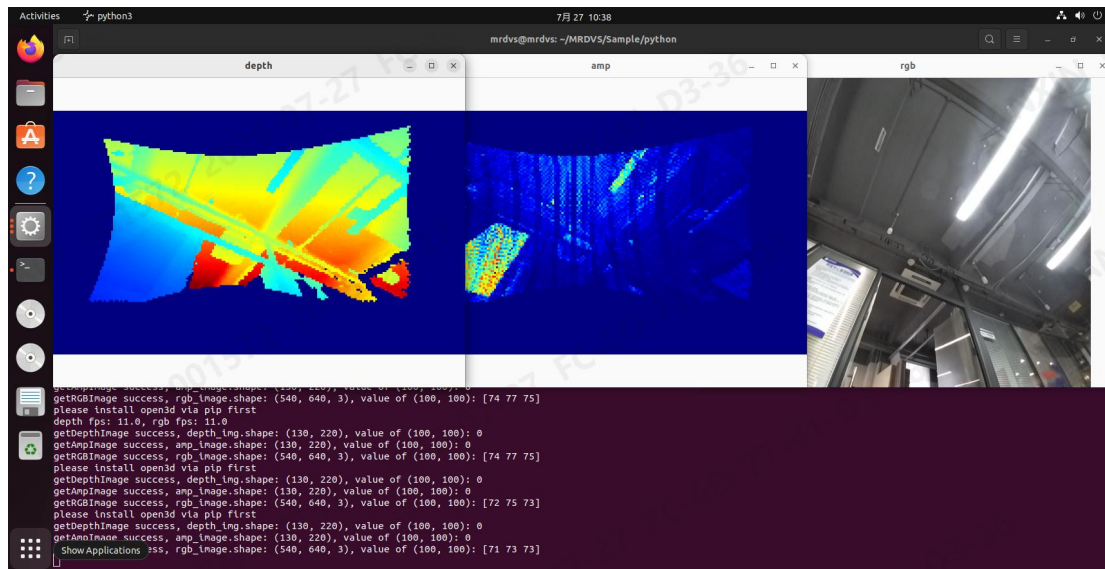
示例会注册设备状态回调，开启深度、强度和 RGB 数据流，读取点云、帧率及图像，并使用 OpenCV 显示窗口。程序没有交互退出键，结束时使用 Ctrl+C。

***无桌面环境：**服务器或 SSH 终端没有图形显示时，OpenCV 窗口会启动失败。此时优先使用 C/C++ 基础示例；若必须使用 Python，应在二次开发代码中移除窗口显示，仅保留取流、保存或数据处理逻辑。

***数据流：**基础 Python 示例默认尝试开启深度、强度和 RGB 流，并可选调用 open3d 显示点云。当前型号不支持某一数据流时，应在示例中关闭对应开关；仅需要点云窗口时再安装 open3d。

***设备选择：**按索引连接适合单相机首次验证。多相机部署或需要稳定连接指定设备时，建议使用实际 IP 地址、设备序列号或设备 ID 进行区分。

***算法示例：**application_obstacle.py 和 application_pallet.py 含固定设备 IP 及算法配置，只适用于支持对应算法的相机，运行前必须改为实际设备和参数。application_location.py 标记为未完成，不作为客户现场直接运行项。



5 ROS1 示例

5.1 安装 ROS Noetic 及依赖

Ubuntu 20.04 未安装 ROS 时，先配置 ROS 软件源并安装 Noetic。ROS Noetic 已于 2025 年 5 月结束官方维护，建议仅用于既有项目兼容。

```
# 以下步骤需要 Ubuntu 能够正常访问互联网
sudo apt install -y curl
# 导入 ROS 软件源签名并添加 ROS1 软件源
```

```
sudo curl -sSL
https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc -o
/usr/share/keyrings/ros-archive-keyring.asc
echo "deb [signed-by=/usr/share/keyrings/ros-archive-keyring.asc]
http://packages.ros.org/ros/ubuntu focal main" | sudo tee
/etc/apt/sources.list.d/ros1.list
# 安装 ROS Noetic 及示例依赖
sudo apt update
sudo apt install -y ros-noetic-desktop-full python3-rosdep
sudo apt install -y ros-noetic-cv-bridge ros-noetic-image-transport ros-
noetic-pcl-ros
sudo apt install -y libpcl-dev libopencv-dev
```

把 ROS 环境加入当前用户配置，并初始化 rosdep：

```
# 配置 ROS1 环境，并使配置对后续终端生效
echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
source /opt/ros/noetic/setup.bash
# 首次使用 rosdep 时初始化；已经初始化可跳过第一行命令
sudo rosdep init
rosdep update
```

***重复初始化：**若 sudo rosdep init 提示配置文件已存在，可直接执行 rosdep update。

5.2 编译并启动普通相机节点

进入 MRDVS 目录后执行：

```
# 进入 ROS1 工作空间并安装缺失依赖
cd Sample/ROS/lx_camera_node_ws
rosdep install --from-paths src --ignore-src -r -y
# 编译工作空间并加载运行环境
catkin_make
source devel/setup.bash
# 启动普通相机节点
roslaunch lx_camera_ros lx_camera_ros.launch
```

另开终端并执行以下命令验证节点、话题和帧率：

```
# 新开终端后重新加载 ROS1 及工作空间环境
source /opt/ros/noetic/setup.bash
cd Sample/ROS/lx_camera_node_ws
source devel/setup.bash
# 检查节点和话题是否已经发布
```

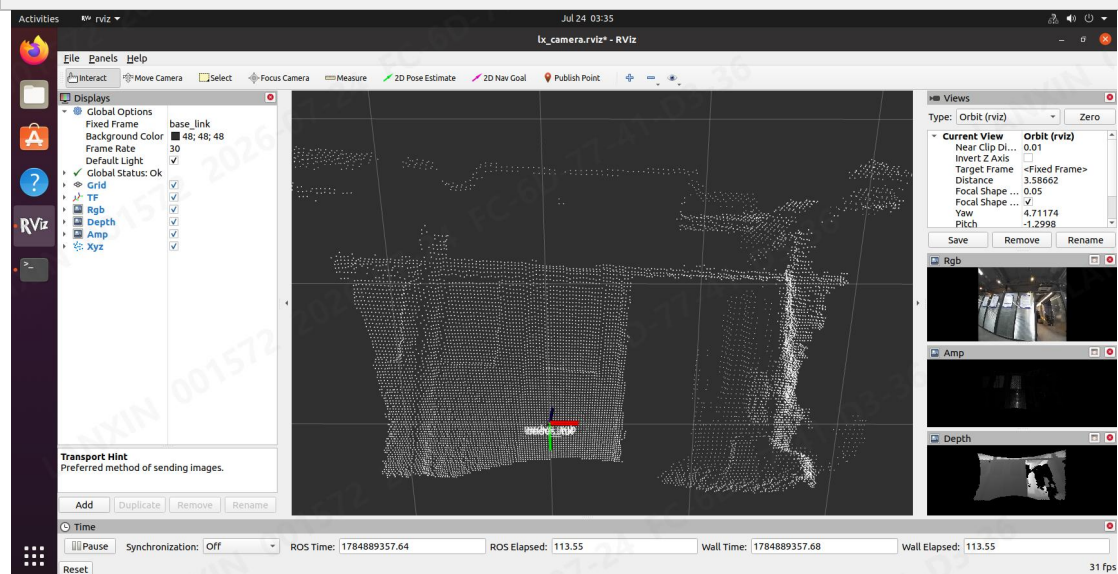
```

rosnode list
rostopic list
# 分别检查深度图、RGB 图和点云的发布频率
timeout 8s rostopic hz /lx_camera_node/LxCamera_Depth
timeout 8s rostopic hz /lx_camera_node/LxCamera_Rgb
timeout 8s rostopic hz /lx_camera_node/LxCamera_Cloud
# 读取一条相机帧率消息
rostopic echo -n 1 /lx_camera_node/LxCamera_FrameRate

```

需要同时打开 RViz 时使用：

```
roslaunch lx_camera_ros lx_camera_ros.launch enable_rviz:=true
```



5.3 launch 文件

文件	用途	使用条件
lx_camera_ros.launch	普通深度、RGB、点云和状态节点。	通用入口，默认按索引 0 打开设备。
lx_lidar_ros.launch	XYZIRT 雷达点云和 IMU 节点。	仅支持雷达点云或 IMU 的型号。
mapping.launch	视觉定位模块建图。	需配置实际 IP、外部传感器话题和外参。
localization.launch	视觉定位。	需配置实际 IP、有效地图名、里程计和外参。
sensor_sim.launch	发布虚拟激光、里程计和激光位姿。	仅用于定位链路联调，不替代真实传感器。
obstacle.launch	旧版避障 V1 配置。	参数名为旧接口；建议在通用 launch 中配置

		LX_INT_ALGORITHM_MODE=1。
obstacleV2.launch	旧版避障 V2 配置。	参数名为旧接口；建议在通用 launch 中配置 LX_INT_ALGORITHM_MODE=4。
pallet.launch	旧版托盘算法配置。	参数名为旧接口；建议在通用 launch 中配置 LX_INT_ALGORITHM_MODE=2。

***算法启动：**普通节点只读取以 LX_开头的当前参数。ROS1 的 lx_camera_ros.launch 已配置 LX_INT_ALGORITHM_MODE，使用内置算法时直接修改其数值；ROS2 的对应参数默认被注释，需要取消注释后设置相应数值。obstacle、obstacleV2 和 pallet 等旧版 launch 仍使用旧参数名，不建议作为当前版本的启动入口。

***辅助脚本：**rate.sh、obstacle.sh 和 pallet.sh 用于订阅对应 ROS1 话题，需在 lx_camera_node_ws 目录运行。build.sh 会清理原 build 和 devel 目录后重新编译；日常增量编译建议直接使用 catkin_make。

5.4 普通节点参数

参数	含义	说明
ip	设备选择	0 或空字符串按索引 0 打开；填写完整 IPv4 地址时按 IP 打开。
log_level / log_path	SDK 日志	默认等级 1，日志目录应对当前用户可写。
is_depth	发布深度图	1 开启，0 关闭。
is_xyz	点云类型	0 关闭；1 发布 XYZ 点云；2 发布含强度和时间信息的雷达点云。
LX_BOOL_ENABLE_3D_AMP_STREAM	3D 强度流	1 开启，0 关闭。
LX_BOOL_ENABLE_2D_STREAM	RGB 流	1 开启，0 关闭。
LX_BOOL_ENABLE_IMU	IMU	1 开启；不支持的型号会返回 LX_W_NOT_SUPPORT。
LX_INT_XYZ_UNIT	点云单位	0 为毫米，1 为米；RViz 通常使用米。
LX_INT_XYZ_COORDINATE	点云坐标系	0 为相机坐标系，1 为机器人坐标系。
x/y/z/yaw/roll/pitch	安装外参	平移单位为米，旋转单位为度。

5.5 话题与服务

话题	类型	内容
LxCamera_Rgb	sensor_msgs/Image	RGB 图像。
LxCamera_Depth	sensor_msgs/Image	深度图。
LxCamera_Amp	sensor_msgs/Image	强度图。
LxCamera_RgbInfo / LxCamera_TofInfo	sensor_msgs/CameraInfo	RGB 和 TOF 相机标定信息。
LxCamera_Cloud	sensor_msgs/PointCloud2	XYZ 或 RGB 点云。
LxCamera_LidarCloud	sensor_msgs/PointCloud2	XYZIRT 雷达点云。
LxCamera_Imu	sensor_msgs/Imu	IMU 数据。
LxCamera_FrameRate	lx_camera_ros/FrameRate	深度、强度、RGB 帧率和温度。
LxCamera_Pallet	lx_camera_ros/Pallet	托盘算法结果。
LxCamera_Obstacle	lx_camera_ros/Obstacle	避障算法结果。

普通节点还提供 LxCamera_LxCmd、LxCamera_LxInt、LxCamera_LxBool、LxCamera_LxFloat 和 LxCamera_LxString 服务，分别用于命令、整数、布尔、浮点和字符串特征的访问。完整名称位于/lx_camera_node 命名空间下。

6 ROS2 示例

6.1 安装 ROS2 Humble 及依赖

ROS2 示例以 Ubuntu 22.04 和 Humble 为目标环境。全新系统可按以下方式安装：

```
# 以下步骤需要 Ubuntu 能够正常访问互联网
sudo apt update
sudo apt install -y software-properties-common curl
sudo add-apt-repository -y universe
# 导入 ROS2 软件源签名并添加 Humble 软件源
sudo curl -sSL
https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
/usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/usr/share/keyrings/ros-archive-keyring.gpg]"
```

```
http://packages.ros.org/ros2/ubuntu jammy main" | sudo tee
/etc/apt/sources.list.d/ros2.list
# 安装 ROS2 Humble、编译工具及示例依赖
sudo apt update
sudo apt install -y ros-humble-desktop python3-colcon-common-extensions
python3-rosdep
sudo apt install -y ros-humble-cv-bridge ros-humble-image-transport
sudo apt install -y ros-humble-pcl-conversions ros-humble-tf2-ros libpcl-
dev libopencv-dev
# 配置 ROS2 环境，并使配置对后续终端生效
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source /opt/ros/humble/setup.bash
# 首次使用 rosdep 时初始化；已经初始化可跳过第一行命令
sudo rosdep init
rosdep update
```

6.2 编译、启动与验证

进入 MRDVS 目录后执行：

```
# 进入 ROS2 工作空间并安装缺失依赖
cd Sample/ROS2/lx_camera_node_ws
rosdep install --from-paths src --ignore-src -r -y
# 编译工作空间并加载运行环境
colcon build
source install/setup.bash
# 启动普通相机节点
ros2 launch lx_camera_ros lx_camera_ros.launch.py
# 普通相机节点默认关闭强度数据流。需要发布强度图时，通过 enable_amp 参数开启
ros2 launch lx_camera_ros lx_camera_ros.launch.py enable_amp:=1
# 需要同时打开 RViz2 时
ros2 launch lx_camera_ros lx_camera_ros.launch.py enable_rviz:=true
```

另开终端验证：

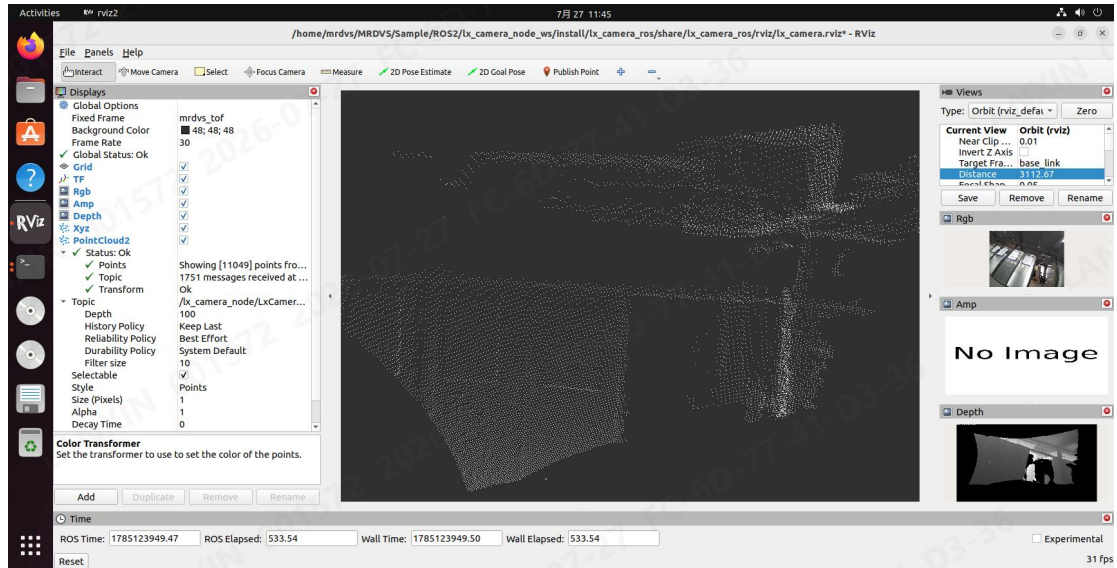
```
# 新开终端后重新加载 ROS2 及工作空间环境
source /opt/ros/humble/setup.bash
cd Sample/ROS2/lx_camera_node_ws
source install/setup.bash
# 检查节点和话题是否已经发布
```

```

ros2 node list
ros2 topic list
# 检查深度图发布频率，观察后按 Ctrl+C 退出
ros2 topic hz /lx_camera_node/LxCamera_Depth
# 读取一条相机帧率消息
ros2 topic echo --once /lx_camera_node/LxCamera_FrameRate

```

***修改 launch：** 应修改 `src/lx_camera_ros/launch` 中的源文件，然后重新执行 `colcon build` 并 `source install/setup.bash`；不要只修改 `install` 目录中的生成副本。



6.3 CycloneDDS

若节点能运行但订阅端帧率明显偏低，可安装 CycloneDDS 并在启动节点的终端中选择该 RMW 实现：

```

# 安装并启用 CycloneDDS 通信实现
sudo apt install -y ros-humble-rmw-cyclonedds-cpp
echo "export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp" >> ~/.bashrc
source ~/.bashrc
# 使用新的 DDS 配置重新启动节点
ros2 launch lx_camera_ros lx_camera_ros.launch.py

```

ROS2 提供 `lx_camera_ros.launch.py`、`lx_lidar_ros.launch.py`、`mapping.launch.py`、`localization.launch.py`、`sensor_sim.launch.py`、`obstacle.launch.py`、`obstacleV2.launch.py` 和 `pallet.launch.py`，功能与 ROS1 对应。使用 `lx_camera_ros.launch.py` 时，普通节点完整名称位于 `/lx_camera_node` 命名空间下，定位节点位于 `/lx_localization_node` 命名空间下。

***辅助脚本：** ROS2 目录中的 `rate.sh`、`obstacle.sh` 和 `pallet.sh` 使用了不带命名空间的话题名；应改用本章给出的 `/lx_camera_node/LxCamera_*` 完整名称。`build.sh` 会清理 `build`、`install` 和 `log` 目录，日常增量编译建议使用 `colcon build`。

7 视觉定位、建图与重定位

视觉定位功能仅适用于已部署对应算法的相机。使用前应确认相机算法版本、地图格式、外部传感器数据和安装外参。ROS1 和 ROS2 均提供 lx_localization_node、mapping、localization 和 sensor_sim 示例。

7.1 输入、输出与关键配置

项目	内容
建图输入	绝对里程计 nav_msgs/Odometry；按场景输入 LaserScan 和激光位姿 PoseStamped。
定位输入	绝对里程计；重定位时输入初始位姿。
定位输出	LxCamera_LocationResult，类型为 geometry_msgs/PoseStamped。
建图模式	mapping_mode=true，localization_mode=false。
定位模式	mapping_mode=false，localization_mode=true，并设置相机中已存在的有效 map_name。
外参	camera_extrinsic_param 格式为[x,y,z,yaw,pitch,roll]；laser_extrinsic_param 格式为[x,y,yaw]。
模拟话题	/sim/scan、/sim/odom、/sim/scan_pose，仅用于链路联调。

7.2 联调顺序

1. 在 mapping.launch 或 localization.launch 中设置实际相机 IP、话题名称、地图名和外参。定位文件中的示例 IP 和地图名不能直接用于其他现场。
2. 启动 sensor_sim，确认模拟激光、里程计和激光位姿话题持续发布。
3. 启动 mapping 节点；状态为 20 表示处于正常建图状态。切换真实传感器前，先确认话题类型、时间戳和坐标系一致。
4. 建图完成后先关闭建图使能，再导出原始数据。离线地图生成和部署按项目交付流程完成。
5. 导入有效地图后启动 localization 节点。发送重定位位姿时不要把 x、y 和 yaw 全部设为 0，否则请求可能被判定为误发。

```
# ROS1: 在终端 1 启动传感器模拟节点
roslaunch lx_camera_ros sensor_sim.launch
# ROS1: 在终端 2 启动建图节点
roslaunch lx_camera_ros mapping.launch
# ROS2: 在终端 1 启动传感器模拟节点
```

```
ros2 launch lx_camera_ros sensor_sim.launch.py
# ROS2: 在终端 2 启动建图节点
ros2 launch lx_camera_ros mapping.launch.py
```

*详细流程：地图导入导出、建图数据交付、重定位消息和全部状态码见软件包内 README_Localization(ROS1).pdf 或 README_Localization(ROS2).pdf。地图后缀可能随相机算法版本变化，应以实际设备要求为准。

8 常见问题

现象	原因与处理方法
搜索不到设备	确认主机与相机同网段、网卡已连接、路由正确且防火墙未阻断 SDK 端口。
Python 提示 file libLxCameraApi.so does not exist	Python 示例仍使用相对动态库路径。按第 4.2 节将路径修改为/opt/MRDVS/lib/libLxCameraApi.so，并运行当前目录中的 single_camera.py。
动态库或依赖库加载失败	执行 sudo ldconfig 刷新缓存，再使用 ldconfig -p 和 ldd /opt/MRDVS/lib/libLxCameraApi.so 检查缺失依赖。
LX_E_CTRL_PERMISS_ERROR	相机被其他进程占用。关闭其他 demo、ROS 节点和上位机，等待控制心跳释放后重试。
LX_W_NOT_SUPPORT	当前相机不支持所设置的 IMU、算法、数据流或参数。按型号能力关闭对应配置。
只有部分图像或点云	检查相机流开关、is_depth、is_xyz 及 LX_BOOL_ENABLE_2D_STREAM、LX_BOOL_ENABLE_3D_AMP_STREAM。
图像窗口无法打开	当前终端没有图形环境或 DISPLAY 不可用。改用无显示数据处理，或在本地图形桌面运行。
ROS 节点启动后很快退出	常见原因是 IP 不匹配、控制权限被占用、日志目录不可写或设备功能配置不支持。先查看启动终端的首个错误。
出现 new node registered with same name	同名 ROS 节点重复启动。先结束旧节点，再重新 roslaunch。
ROS2 话题帧率偏低	切换 CycloneDDS，并确认订阅端 QoS、CPU 负载和网络带宽。
CMake 显示 PCL 目录链接警告	当前工程把部分 PCL 包含目录作为链接项，若最终生成节点且运行正常可记录该警告；编译失败时检查 PCL 和 Eigen 安装。

Ctrl+C 后立即无法重启	示例未正常执行关闭流程，确认进程已结束并等待设备释放；生产代码应增加信号处理和资源清理。
----------------	--

完成基础验证后，建议以 `single_camera2` 或普通 ROS 节点为最小可运行基线，再逐项开启 RGB、强度、IMU、雷达点云和内置算法。一次只增加一项配置，便于区分设备能力、网络问题和软件配置问题。